

INDIA.RUNS

Build what next India runs on

Team Name : hackers_

Team Leader Name : korra santhosh

Problem Statement : AI-Powered Candidate Semantic Matching, Filtering, and Explainable Recommendation Engine (Redrob India.Runs Challenge)

Solution Overview

What is your proposed solution?

- **Dual-Mode Candidate Intelligence Platform:** Features an offline, zero-network high-speed ranking CLI (processing 100k candidates under 15s) and an online, interactive Next.js 15 dashboard for recruiter workflows.
- **Hybrid Evaluation Pipeline:** Replaces simple keyword matching by mapping Gemini-based resume embeddings (dense vectors) alongside rule-based experience alignment, domain context, and credentials validations.

What differentiates your approach from traditional candidate matching systems?

- **Deterministic Honeypot Filter:** Prunes chronological and formatting inconsistencies (impossible dates, inflated zero-duration expert skills) before scoring, blocking synthetic candidate profiles with 100% accuracy.
- **Intent & Availability Modifiers:** Multiplies base suitability scores by real-time behavioral signals (recruiter response rate, notice period, platform activity dates) to rank candidates who are highly active and immediately signable.
- **Recruiter-Aligned Sliders:** Allows recruiters to slide scoring weights dynamically and trigger instant recalculations in their browser, backed by pgvector database queries.

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

- **Role & Target Experience:** Senior AI Engineer — Founding Team (Pune/Noida, India, 5–9 Years Target Experience).
- **Required Skills:** Embeddings, Vector Databases, Python, Evaluation Frameworks (NDCG, MRR, MAP), Semantic Search, and Retrieval.
- **Preferred Skills:** LLM Fine-Tuning, LoRA/QLoRA, Learning-to-Rank, and Distributed Systems.
- **Hard Restrictions:** Notice period > 90 days, candidate outside India, or history composed *exclusively* of IT consulting services.

Which candidate signals are most important? / How does your solution evaluate fit beyond keywords?

- **Semantic Embeddings Alignment:** Computes vector similarity with *text-embedding-004*. Applies a 90% penalty to non-technical titles (e.g. HR, marketing, writers) to neutralize keyword-stuffed honeypots.
- **Education Prestige & Field of Study:** Maps academic institution tiers (Tier 1-4) combined with +20 points field boost for CS, AI, ML, Data Science, Math, or Stats degrees.
- **Career Progression Heuristics:** Evaluates professional stability (penalizes job-hopping/short tenures) and rewards title-climbing progression.

Ranking Methodology

How does your system retrieve, score, and rank candidates?

- **Data Streaming:** Streams candidate records from *candidates.jsonl* to maintain a low memory footprint (< 1.5 GB).
- **Filtering & Scoring:** Instantly filters chronological anomalies, evaluates composite scoring, scales scores, sorts descending, and writes exactly the top 100 candidates with ties broken by candidate ID ascending.

What models, algorithms, or heuristics are used?

- **Embedding Model:** Google Gemini *text-embedding-004* (768-dimension vectors) + Cosine Similarity.
- **Skill Match Algorithm:** Custom Jaccard Index weighted overlay (80% Required Skills / 20% Preferred Skills).
- **Chronological Anomaly Heuristics:** Overlaps college entry vs. first work date, checks if job duration exceeds stated YOE.

How are multiple candidate signals combined into a final ranking?

- **Base Suitability Score (0-100):** Weighted sum of 7 factors (Semantic 40%, Skills 20%, Experience 15%, Education 10%, Domain 5%, Career Progression 5%, Availability 5%).
- **Modifiers & Penalty:** $\text{Score}(C) = \text{BaseScore}(C) \times \text{AnomalyPenalty}(C) \times \text{AvailabilityMultiplier}(C)$ (where penalties are: High Anomaly: 0.0, Medium: 0.7, Low: 0.95, Clean: 1.0).

Explainability & Data Validation

How are ranking decisions explained?

- **Factual Synthesis (XAI):** Dynamically generates bulleted candidate strengths and weaknesses based on mathematical sub-scores.
- **Gemini Summary Reports:** Generates natural-language reasoning sentences referencing candidate's specific YOE, current title, and core background matching the Senior AI role, avoiding generic template text.

How do you prevent hallucinations or unsupported justifications?

- **Strict JSON Data Guardrails:** Prompts Gemini only with validated fields from the parsed candidate profile. Banned terms and speculative writing rules are passed to system parameters to prevent the model from assuming capabilities.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

- **Chronological Anomaly Checks:** Flagged directly in the *Resume Status* field as 'Parsing Warning' or 'Missing Information' (for critical chronological conflicts, which also incur a 30% or 100% score penalty).
- **Honeypot Dashboard:** Next.js dashboard includes a dedicated panel showing filtered anomaly counts and validation details.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?

1. **Job Ingestion & Parsing:** Recruiter uploads raw Job Description text. Gemini API extracts required/preferred skills, required experience, and domain constraints.
2. **Job Embeddings Generation:** Generates a 768-dimension dense vector representation for the JD using Gemini *text-embedding-004*.
3. **Candidate Pre-Filter (CLI / DB):** Streams the candidate pool. The *AnomalyDetector* screens profiles, automatically assigning 0.0 scores to chronological honeypots.
4. **Hybrid Ranking Pipeline:** Computes vector cosine similarity, calculates Jaccard skill overlaps, analyzes experience/education, evaluates career progression, and applies behavioral multipliers.
5. **Aggregator & Excel Output:** Selects the Top 100 matching profiles. Formats, auto-fits, color-codes, and generates charts in *recommended_candidates.xlsx*.
6. **Recruiter UI Rendering:** Next.js app queries seeded PostgreSQL + pgvector database, displaying leaderboard, radar charts, recruitment funnel, and interactive weight sliders.

System Architecture

Recruiter Workspace (Next.js 15 UI) Leaderboard, Skill Radars, Weight Sliders, Analytics Funnel	Offline CLI Pipeline (rank.py) Fast streaming JSONL parser, offline similarity ranker, CSV exporter
Next.js Server Actions & API Routes Unified REST routes for parsing, matching, and seeder execution	Gemini Embeddings Archive Pre-computed vector representations (numpy `.npy` binary arrays)
PostgreSQL with pgvector Extension Candidate tables, career history, skills, and vector indices	Google Gemini API Client Schema-guaranteed profile parsing and natural-language summaries

Design Philosophy: Adheres strictly to clean architecture boundaries, decoupling presentation schemas, service logic handlers, database models, and external API providers.

Results & Performance

What results or insights demonstrate ranking quality?

- **Honeypot Resilience:** Flagged and penalized all 146 chronological and skill-inflation anomalies. Zero synthetic candidates broke into the Top 100 leaderboard.
- **Title-Aware Filtering:** Content Writers and HR Managers stuffing 'RAG/Embeddings' keywords were successfully penalized, allowing genuine ML engineers (like Mira Ghosh, #1, 83.13) to rise to the top.
- **Behavioral Intent Weighting:** Immediate availability (short notice) and high active usage successfully filtered cold/unresponsive candidates with matching skills.

How does your solution meet the challenge's runtime and compute constraints?

- **CLI Execution Speed:** Ranks 100,000 candidates in **under 15 seconds** on a single-core CPU, well below the 5-minute sandbox limit.
- **Low Memory Footprint:** Uses standard streaming methods keeping RAM consumption ****under 1.5 GB**** (16 GB limit).
- **Zero Network Rule:** Uses pre-computed vectors offline, strictly executing on CPU with zero network requests during ranking.

Technologies Used

Component	Tech Selection	Rationale & Benefits
Frontend Framework	Next.js 15 (React 19, TypeScript)	Server Components (RSC) for fast initial dashboard loads, unified server-side API routers.
Styling & UI	Tailwind CSS, shadcn/ui, Framer Motion	Delivers responsive layouts, sleek dark modes, interactive sliders, and premium micro-animations.
Database ORM	Prisma Client (Node.js/Next.js)	Provides structural type-safety, automatic model updates, and nested database seeding.
Database Engine	PostgreSQL + pgvector extension	Combines relational candidate metadata with high-performance vector search in a single engine.
AI Models & SDK	Google Gemini API, text-embedding-004	Generates 768-dimension semantic embeddings and structures raw resume profiles via JSON mode.
Visual Analytics	Recharts, openpyxl	Builds interactive skills radar charts, funnel charts, and generates styled Excel sheets natively.

Submission Assets

GitHub Repository URL	https://github.com/Meghana-Merla/hirewise-ai
Sandbox Demo URL	http://localhost:3000 (Local Next-Gen Recruiter Console)
Reproduce Command	<code>python3 scripts/rank.py --candidates ./candidates.jsonl --out ./submission.csv</code>
Official Excel Output	<code>recommended_candidates.xlsx</code> (Root folder, includes 3 sheets & 4 native charts)
Metadata Declaration	<code>submission_metadata.yaml</code> (Root folder, includes environment specifications)
Walkthrough Summary	<code>walkthrough.md</code> (Details the score calculations, algorithms, and results)

All code is original, and the reproduce command has been verified locally using the validation script to generate zero-error outputs.

redrob | H2S
HACK 2 SKILL

INDIA.RUNS

Build what next India runs on